

Unit-3

Process Synchronization

Contents

- **Process Synchronization** - The Critical Section Problem: Solution to Critical Section Problem (Software and Hardware), Synchronization Hardware, Classical Problems of Synchronization, Semaphores, Monitors.
- **Inter process Communication Mechanisms:** IPC between processes on a single computer system, IPC between processes on different systems, using pipes, FIFOs, message queues, shared memory.

Process Synchronization

- Process Synchronization is the coordination of execution of multiple processes in a multi-process system to ensure that they access shared resources in a controlled and predictable manner
- It aims to resolve the problem of race conditions and other synchronization issues in a concurrent system.
- Concurrent access to shared data may result in ***data inconsistency***
- To achieve this, various synchronization techniques such as semaphores, monitors, and critical sections are used.

Process

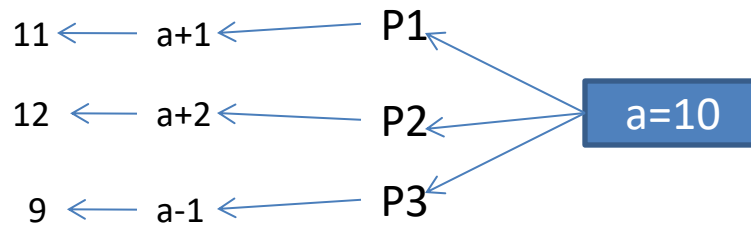
- A process is a program that is currently running or a program under execution is called a process.

Types of Process

- On the basis of synchronization, processes are categorized as one of the following two types:
- **Independent Process** : The execution of one process does not affect the execution of other processes.
- Here, no shared memory
- Here, process synchronization not required
- **Cooperative Process** : one process execution affects the execution of another process.
- Here, multiple process shared same memory
- Here, process synchronization required

Race Condition

- When more than one process is executing the same code or accessing the same memory or any shared variable in that condition there is a possibility that the output or the value of the shared variable is wrong so for that all the processes doing the race to say that my output is correct this condition known as a race condition.
- Example:



- In above example process P1,P2,P3 accessing same memory that is `a=10`.
- According P1,P2,P3, p1 result is 11, p2 result is 12, p3 result is 9
- Here , which one is correct value.
- P1 says that `a=11` is correct value, P2 says that `a=12`, P3 says that a value is 9 this is called race condition.
- For every process value will be changed that I called inconsistent data.
- Making of inconsistent data to consistent data we are moving process synchronization.

Race Condition

- ❓ “count++” may be implemented in machine language (on typical machine) as:

```
register1 = count  
register1 = register1 + 1  
count = register1
```

- ❓ “count--” may be implemented in machine language (on typical machine) as:

```
register2 = count  
register2 = register2 - 1  
count = register2
```

- ❓ Consider this execution interleaving with “count = 5” initially:

T0: producer execute `register1 = count` {register1 = 5}

T1: producer execute `register1 = register1 + 1` {register1 = 6}

T2: consumer execute `register2 = count` {register2 = 5}

T3: consumer execute `register2 = register2 - 1` {register2 = 4}

T4: producer execute `count = register1` {count = 6 }

T5: consumer execute `count = register2` {count = 4}

- Notice that we have arrived at the incorrect state “counter == 4”, indicating that four buffers are full, when, in fact, five buffers are full. If we reversed the order of the statements at *T4 and T5*, we would arrive at the incorrect state “counter == 6”.
- We would arrive at this incorrect state because we allowed both processes to manipulate the variable counter concurrently.
- A situation like this, where **several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place**, is called a *race condition*.
- Clearly , we want the resulting changes not to interfere with one another . Hence we need *process synchronization*.

Critical Section Problem

- *Critical Section*: Section of Code that accesses Some Shared Data in a mutually exclusive manner
- Consider System of n Processes $\{p_0, p_1, \dots p_{n-1}\}$
- Each process has a segment of code, called a critical section Process may be Changing
 - Common Variables, Updating table, Writing file, etc.
- The important feature of the system is that, when one process is executing in its critical section, no other process is allowed to execute in its critical section.
- That is, no two processes are executing in their critical sections at the same time.
- The ***critical-section problem is to design a*** protocol that the processes can use to cooperate.
- Each process must request permission to enter its critical section. The section of code implementing this request is the **entry section**.
- The critical section may be followed by an **exit section**.
- The remaining code is the **remainder section**.

General Structure

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (true);
```

Solution to Critical-section Problem Using Locks

```
do {  
    acquire lock  
    critical section  
    release lock  
    remainder section  
} while (TRUE);
```

The general structure of a typical process P_i is shown in Figure.

A Critical Section Environment Contains:

Entry Section	Code requesting entry into the Critical Section.
Critical Section	Code in which Only One Process can execute at any one time.
Exit Section	End of the Critical Section, releasing or allowing others in.
Remainder Section	Rest of the Code AFTER the Critical Section.

Solution to Critical–Section Problem

1. Mutual Exclusion

- ✓ If process P_i is executing in its Critical Section, then No Other Processes can be executing in the Critical Sections.

2. Progress

- ✓ If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely.

3. Bounded Waiting

- ✓ A bound must exist on the No. of times that Other processes are allowed to enter the Critical Sections after a process has made a request to enter its Critical Section and before that request is granted

Peterson's Solution

- It is a Classic software-based solution to the critical-section problem known as **Peterson's solution**.
- **Because of the way modern computer** architectures perform basic machine-language instructions, such as load and store, there are no guarantees that Peterson's solution will work correctly on such architectures.
- However, we present the solution because it provides a good algorithmic description of solving the critical-section problem and illustrates some of the complexities involved in designing software that addresses the requirements of mutual exclusion, progress, and bounded waiting.
- Peterson's solution is restricted to two processes that alternate execution between their critical sections and remainder sections.
- The processes are numbered *P0* and *P1*. For convenience, when presenting *Pi* , we use *Pj* to denote the other process.
- Peterson's solution requires the two processes to share two data items:
 - ✓ int turn;
 - ✓ boolean flag[2];

Peterson's Solution

- Good algorithmic description of solving the problem
- **Two process solution, say i and j**
- The two processes share two variables:
 - **int turn;**
 - **Boolean flag[2]**
- The variable **turn** indicates whose turn it is to enter the critical section
- The **flag** array is used to indicate if a process is ready to enter the critical section. **flag[i] = true** implies that process **P_i** is ready!

Peterson solution requires two data items to be shared between the two processes.

`int turn`

→ Indicates whose turn it is to enter its critical section.

Structure of process P_i in Peterson's solution

```
do {  
    flag [i] = true ;  
    turn = j ;  
    while ( flag [ j ] && turn == [ j ] ) ;
```

critical section

```
    flag [i] = false ;
```

remainder section

```
    } while (TRUE) ;
```

`boolean flag [2]`

→ Used to indicate if a process is ready to enter its critical section.

Structure of process P_j in Peterson's solution

```
do {  
    flag [ j ] = true ;  
    turn = i ;  
    while ( flag [ i ] && turn == [ i ] ) ;
```

critical section

```
    flag [ j ] = false ;
```

remainder section

```
    } while (TRUE) ;
```

Peterson's Solution (Cont.)

- Provable that the three CS requirement are met:
 1. Mutual exclusion is preserved

P_i enters CS only if:
either **flag[j] = false** or **turn = i**
 2. Progress requirement is satisfied
 3. Bounded-waiting requirement is met

Peterson's Solution (Cont.)

- To enter the critical section, process P_i first sets $flag[i]$ to be true and then sets $turn$ to the value j , thereby asserting that if the other process wishes to enter the critical section, it can do so.
- If both processes try to enter at the same time, $turn$ will be set to both i and j at roughly the same time. Only one of these assignments will last; the other will occur but will be overwritten immediately.
- The eventual value of $turn$ determines which of the two processes is allowed to enter its critical section first.
- We now prove that this solution is correct. We need to show that:
 - **1. Mutual exclusion is preserved.**
 - **2. The progress requirement is satisfied.**
 - **3. The bounded-waiting requirement is met.**

Peterson's Solution (Cont.)

- To prove property 1, we note that each P_i enters its critical section only if either $\text{flag}[j] == \text{false}$ or $\text{turn} == i$. Also note that, if both processes can be executing in their critical sections at the same time, then $\text{flag}[0] == \text{flag}[1] == \text{true}$.
- These two observations imply that P_0 and P_1 could not have successfully executed their while statements at about the same time, since the value of turn can be either 0 or 1 but cannot be both.
- Hence, one of the processes —say, P_j —must have successfully executed the while statement, whereas P_i had to execute at least one additional statement (“ $\text{turn} == j$ ”). However, at that time, $\text{flag}[j] == \text{true}$ and $\text{turn} == j$, and this condition will persist as long as P_j is in its critical section; as a result, mutual exclusion is preserved.

Synchronization Hardware

- To solve critical section problem in two ways:
 - Software(Peterson's Solution)
 - Hardware also known as Test and set lock
- A hardware solution to the synchronization problem there is a shared lock variable which can take either of the two values, i.e., 0 or 1
 - 0 means critical section is not locked
 - 1 means critical section is locked
- Before entering into the critical section, a process inquires about the lock.
- If it is locked, it keeps on waiting till it becomes free.
- If it is not locked, it takes the lock and executes the critical section

```
do {  
    acquire lock  
        critical section  
    release lock  
        remainder section  
} while (TRUE);
```

Synchronization Hardware

- Many systems provide hardware support for implementing the critical section code.
- All solutions below based on idea of **locking**
 - Protecting critical regions via locks
- Uniprocessors – could disable interrupts
 - Currently running code would execute without preemption
 - Generally too inefficient on multiprocessor systems
 - Operating systems using this not broadly scalable
- Modern machines provide special atomic hardware instructions
 - **Atomic** = non-interruptible
 1. Either test memory word and set value
 2. Or swap contents of two memory words

TestAndSet() Instruction

Definition :

```
Boolean TestAndSet(boolean *target){
```

```
Boolean rv= *target;
```

```
*target=True;
```

```
Return rv;
```

```
}
```

1. Executed atomically
2. Returns the original value of passed parameter
3. Set the new value of passed parameter to “TRUE”.

Solution using test_and_set()

✓ Shared Boolean variable lock, initialized to FALSE

✓ Solution:

```
do {  
    while (test_and_set(&lock))  
        ; /* do nothing */  
        /* critical section */  
    lock = false;  
        /* remainder section */  
} while (true);
```

TestAndSet() Example

- Here we have two process P1,P2 in both the processes P1 wants to execute first.
- Lock value always set '0' initially

P1 Execution:-

```
do {  
    while (test_and_set(&lock))  
        ; /* do nothing */  
        /* critical section */  
        *target=True;  
        lock = false;  
        /* remainder section */  
    } while (true);
```

- P2 Execution:

At the same time P2 also try to enter in critical section,

In Critical section already P1 is Executing in critical section. How it will enter

- Here, lock variable is 1 that set by P1

- i.e., *Target=True

```
do {  
    while (test_and_set(&lock))  
        ; /* do nothing */  
        /* critical section */  
        lock = false;  
        /* remainder section */  
    } while (true);
```

Bounded-waiting Mutual Exclusion with TestAndSet()

do {

 waiting[i] = TRUE;

 key = TRUE;

 while (waiting[i] && key)

 key = TestAndSet(&lock);

 waiting[i] = FALSE;

 // critical section

 j = (i + 1) % n;

 while ((j != i) && !waiting[j])

 j = (j + 1) % n;

 if (j == i)

 lock = FALSE;

 else

 waiting[j] = FALSE;

 } while (TRUE);

Common data structures are

boolean waiting[n];

boolean lock;

Initialized to FALSE

Semaphores

- It is proposed by Edsger Dijkstra
- It is a synchronization tool
- To prevent race condition semaphore is a one of the technique.
- **Definition:** It is an integer variable, which is used in a mutual exclusive manner by various concurrent cooperative processes in order to achieve synchronization.
- Does not require busy waiting
- Two standard operations modify S: wait() and signal()
 - Originally called P() and V()
 - P (from the Dutch proberen, “to test”)
 - V (verhogen, “to increment”)
- In entry
- P(), down(), wait()
- In exit
- V(), up(), signal()

Semaphores

- **Definition of Wait():**

- **wait (S) {**
 while S <= 0
 ; // no-op
 S--;
}

- **Definition of Signal():**

- **signal (S) {**
 S++;
}

Note : all the modifications to the integer value of the semaphore in the wait() and signal() operations must be executed indivisibly. That is , when one process modifies the semaphore value, no other process can simultaneously modify that semaphore value.

Semaphore Types

1. Counting Semaphore

- Integer value can range over an unrestricted domain. it is used to control access to a resource that has multiple instances.

2. Binary Semaphore

- The value of a binary semaphore can range only between 0 and 1.
- Can be simpler to implement
- Also known as *mutex locks*
- Advantages :
 1. Provides *mutual exclusion*
 2. Can solve various *synchronization problems*

- 1. Provides Mutual Exclusion- General Structure of a Process

```
– do{  
– Semaphore S=1;  
– wait (S);  
  //Critical Section  
  signal (S);  
  //Remainder Section  
}while(true)
```

2. Can solve various synchronization problems

Consider P_1 and P_2 that require S_1 to happen before S_2

Create a semaphore “**synch**” initialized to 0

P1: //synch=0

S_1 ;

signal (synch) ;

P2:

wait (synch);

S_2 ;

Can implement a counting semaphore S as a binary semaphore

Semaphore Implementation

- Must guarantee that no two processes can execute `wait ()` and `signal ()` on the same semaphore at the same time
- Thus, implementation becomes the critical section problem where the wait and signal code are placed in the critical section
 - Could now have **busy waiting** in critical section implementation
 - But implementation code is short
 - Little busy waiting if critical section rarely occupied
- Note that applications may spend lots of time in critical sections and therefore this is not a good solution

Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
 - value (of type integer)
 - pointer to next record in the list
- Two operations:
 - **block** – place the process invoking the operation on the appropriate waiting queue
 - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue

Semaphore Implementation with no Busy waiting (Cont.)

Struct Semaphore

{ int value; //initially 1

Struct process *list; }

- Implementation of wait:

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0)  
    {  
        add this process to S->list;  
        block();  
    }  
}
```

- Implementation of signal:

```
signal(semaphore *S){  
    S->value++;  
    if (S->value <= 0)  
    {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```

General Structure of a process

```
do{  
    wait (S);  
    //CS  
    signal (S);  
    //RS  
}while(true)
```

Deadlock and Starvation

- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let s and q be two semaphores initialized to 1

P_0

wait (S);
wait (Q);

·
signal (S);
signal (Q);

P_1

wait (Q);
wait (S);

·
signal (Q);
signal (S);

- **Starvation** – indefinite blocking
 - A process may never be removed from the semaphore queue in which it is suspended
- **Priority Inversion** – Scheduling problem when lower-priority process holds a lock needed by higher-priority process
 - Solved via **priority-inheritance protocol**

Classical Problems of Synchronization

- Classical problems used to test newly-proposed synchronization schemes
 - **Bounded-Buffer Problem**
 - **Readers and Writers Problem**
 - **Dining-Philosophers Problem**

Bounded Buffer Problem

- It is also called as produce- Consumer problem.
- Two processes share a common, fixed-size buffer. One of them, the producer, puts information into the buffer, and the other one, the consumer, takes it out.
- There is a buffer of 'n' slots and each slot is capable of storing one unit of data.
- Here, the assumption is both processes are coming at a same time and they are sharing common buffer etc.,



- Here we are talking about cooperative processes, in this two or more processes.. They are sharing something it could be code or resources or memory or some variables also.

Conditions:

Producer side:

- Before placing product into buffer producer have to check buffer is not full.
- If buffer is full producer can't place product and have to wait some time if buffer is have space then place the products.

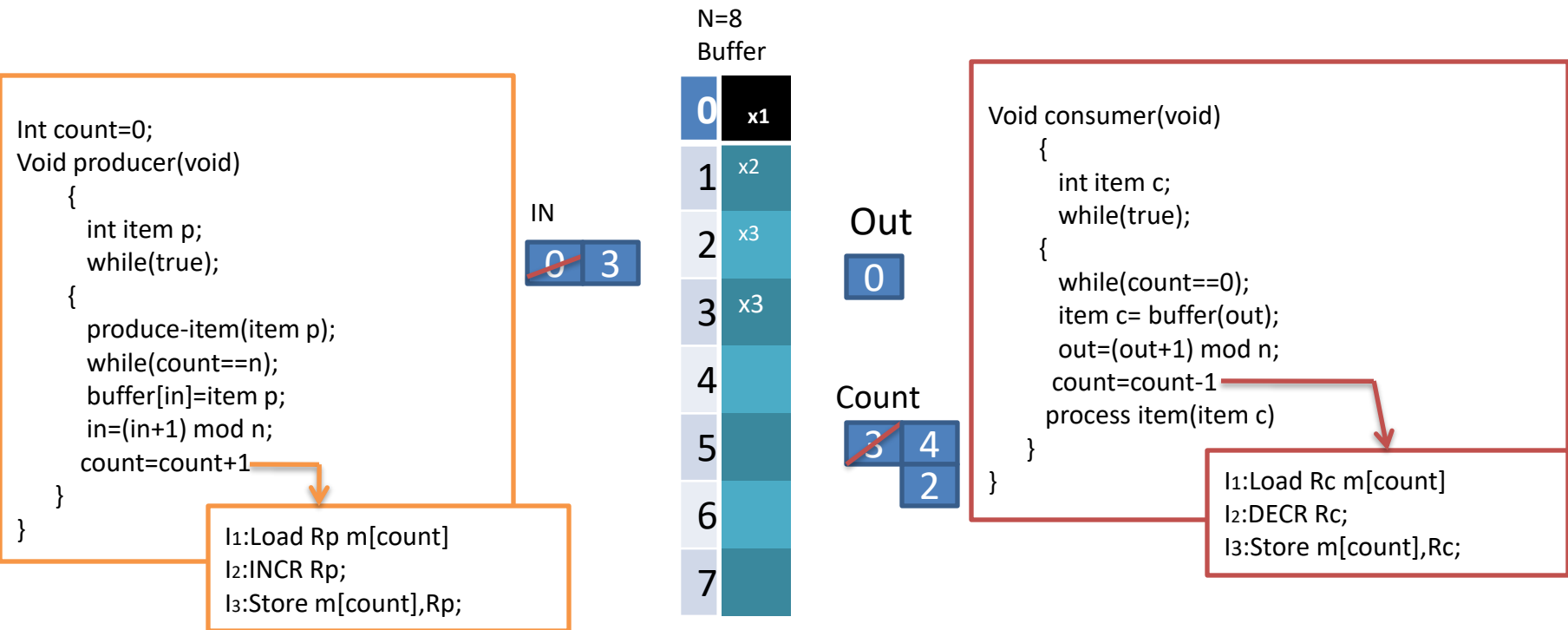
Consumer side:

- Before consuming product from buffer consumer have to check buffer is not empty.
- If buffer empty consumer can't get the product from the buffer.
- The count will be decremented.
- The count is nothing but shared variable.
- Whenever the producer placed the product into the buffer immediately the count will be incremented by 1.
- Whenever the consumer, consumes the product from the buffer immediately the count will be decremented.

Actual problem is:

- The producer tries to insert data into an empty slot of the buffer.
- The consumer tries to remove data from filled slot in the buffer.
- First problem is, the producer must not insert data when the buffer is full.
- Second problem is, the consumer must not remove data when the buffer is empty.
- Last problem is, the producer and consumer should not insert and remove data simultaneously.

produce- Consumer problem.



- Producer place one more product in the buffer for this producer code will be run after completion I1,I2 preempted. And that state saved in PCB. Here, count value is 3 only because it got preempted.
- Then control goes to consumer it will run I1,I2 got preempted, that state saved in PCB. Here, count value is 3 only because not store the count value.
- immediately producer code will be execute it will resume producer code from PCB here only I3 pending that instruction3 will be executed and terminated. now the count value is 4. then control goes to the consumer it will resume the consumer code from PCB. Here count value is 3.
- it execute the intruction3 successfully and terminated. Now the count value is 2.
- Here there is a inconstant data we got that is producer count value is 4 and consumer count value is 2 in these both which one is correct.
- For this inconstant data we need to do synchronization. This is the problem in producer and consumer problem.

Solution to the Bounded Buffer Problem Using semaphore

- We will make use of the three Semaphores:
- **empty**: Counts the number of empty slots in the buffer. Initialized to the size of the buffer N .
- **full**: Counts the number of filled slots in the buffer. Initialized to 0.
- **mutex**: A binary semaphore (or mutex) which is used to acquire and release the lock.

➤ Bounded Buffer Problem (Cont.)

- N buffers, each can hold one item

Initialization

Semaphore **mutex** = 1 , Semaphore **full** = 0, Semaphore **empty** = N

Structure of the producer process

```
do {  
    // produce an item in nextp  
    down (empty);  
    down (mutex);  
    // add the item to the buffer  
    up (mutex);  
    up(full);  
} while (TRUE);
```

Structure of the consumer process

```
do {  
    down (full);  
    down (mutex);  
    // remove an item from buffer to  
    nextc  
    up (mutex);  
    up (empty);  
    // consume the item in nextc  
} while (TRUE);
```

Readers-Writers Problem

- A database is to be shared among several concurrent processes.
- Some of these processes may want only to read the database, whereas others may want to update(that is read and write) the database.
- We distinguish between two types of processes by referring to the formers as readers and to the latter as writers.
- Problem:
- Obviously, if two readers access the shared data simultaneously , no adverse affects will result.
- Here, two processes are reading simultaneously no problem occur in database.
- However, if a writer and some other thread(either reader or a writer)

➤ Readers-Writers Problem

- A data set is shared among a number of concurrent processes
 - Readers – only read the data set; they do **not** perform any updates
 - Writers – can both read and write
- Problem – allow multiple readers to read at the same time
 - Only one single writer can access the shared data at a time
- Several variations of how readers and writers are treated – all involve priorities
- Shared Data
 - Data set
 - Semaphore **mutex** initialized to 1
 - Semaphore **wrt** initialized to 1
 - Integer **readcount** initialized to 0

Readers-Writers Problem (Cont.)

- The structure of a writer process

```
do {  
    wait (wrt) ;  
  
    //  writing is performed  
  
    signal (wrt) ;  
} while (TRUE);
```

Readers-Writers Problem (Cont.)

- The structure of a reader process

do {

Entry Section

```
wait (mutex) ; //Ensure that no other reader can execute the <Entry> section while you are in it
readcount ++ ; //Indicate that you are a reader trying to enter the Critical Section
if (readcount == 1) //Checks if you are the first reader trying to enter CS
    wait (wrt) ; //If you are the first reader, lock the resource from writers. Resource stays
                reserved for subsequent readers
signal (mutex) //Release <Entry> Section. Let other readers enter the <Entry> section, now that
               you are done with it.
// reading is performed – Critical Section
```

Exit Section

```
wait (mutex) ; //Ensure that no other reader can execute the <Exit> section while you are in it
readcount -- ; //Indicate that you are no longer needing the shared resource. One less readers
if (readcount == 0) //Checks if you are the last (only) reader who is reading the shared file
    signal (wrt) ; //If you are last reader, then you can unlock the resource. This makes it available
                  to writers.
signal (mutex) ; //Let other readers enter the <Exit> section, now that you are done with it.
while (TRUE);
```

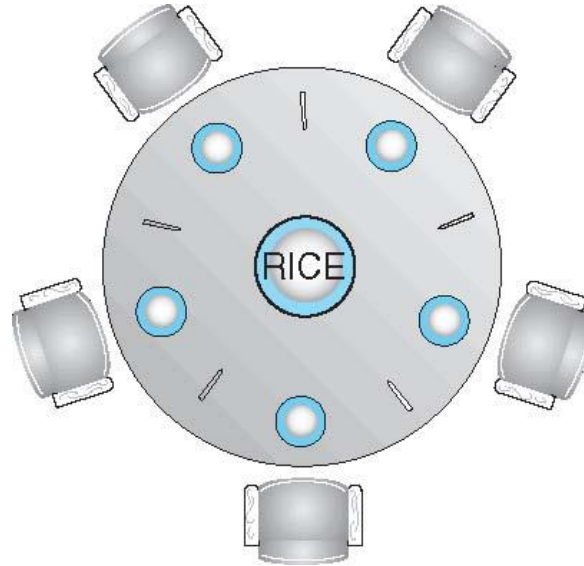

Readers/Writers problem's solution

3. Once the first reader is in the entry section, it will lock the resource. Doing this will prevent any writers from accessing it. Subsequent readers can just utilize the locked (from writers) resource. The very last reader (indicated by the readcount variable) must unlock the resource, thus making it available to writers.
4. In this solution, every writer must claim the resource individually. This means that a stream of readers can subsequently lock all potential writers out and starve them. This is so, because after the first reader locks the resource, no writer can lock it, before it gets released. And it will only be released by the very last reader. Hence, this solution does not satisfy fairness.

➤ Readers-Writers Problem Variations

- *First* variation – no reader kept waiting unless writer has permission to use shared object
- *Second* variation – once writer is ready, it performs write asap
- Both may have starvation leading to even more variations
- Problem is solved on some systems by kernel providing reader-writer locks

➤ Dining-Philosophers Problem



- Philosophers spend their lives thinking and eating
- Don't interact with their neighbors, occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
 - Need both to eat, then release both when done
- In the case of 5 philosophers
 - Shared data
 - Bowl of rice (data set)
 - Semaphore **chopstick** [5] initialized to 1

Dining-Philosophers Problem Algorithm

- The structure of Philosopher i :

```
do {  
    wait ( chopstick[i] );  
    wait ( chopstick[ (i + 1) % 5] );  
  
    // eat  
  
    signal ( chopstick[i] );  
    signal ( chopstick[ (i + 1) % 5] );  
  
    // think  
  
} while (TRUE);
```

- What is the problem with this algorithm?

Problems with Semaphores

- Incorrect use of semaphore operations:
 - `signal (mutex) .... wait (mutex)`
 - `wait (mutex) ... wait (mutex)`
 - Omitting of `wait (mutex)` or `signal (mutex)` (or both)
- Deadlock and starvation

Difference between Lock, Mutex and Semaphore

- A **lock** allows only one thread to enter the part that's locked and the **lock** is not shared with any other processes.
- A **mutex** → **locking mechanism mechanism** used to synchronize access to a resource. Only one task can acquire the mutex. It means there is ownership associated with mutex, and only the owner can release the lock (mutex).
- A **semaphore** does the same as a **mutex** but allows x number of threads to enter.

Semaphore is **signaling mechanism** (“I am done, you can carry on” kind of signal). For example, if you are listening songs (assume it as one task) on your mobile and at the same time your friend calls you, an interrupt is triggered upon which an interrupt service routine (ISR) signals the call processing task to wakeup.

❖ Monitors

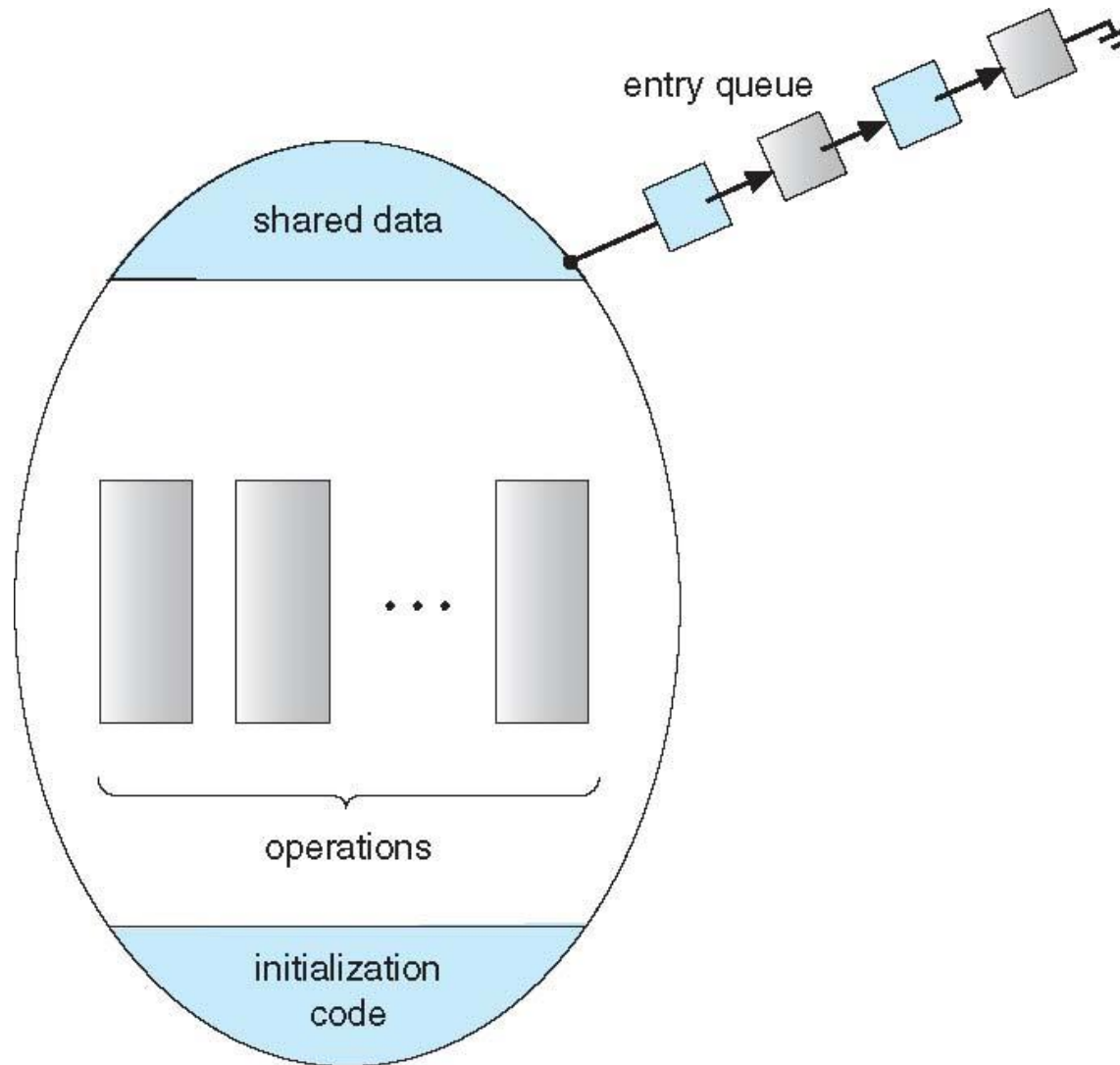
- A high-level abstraction that provides a convenient and effective mechanism for process synchronization
- *Abstract data type*, internal variables only accessible by code within the procedure
- Only one process may be active within the monitor at a time

```
monitor monitor-name
{
    // shared variable declarations
    procedure P1 (...) { .... }

    procedure Pn (...) {.....}

    Initialization code (...) { ... }
}
}
```

Schematic view of a Monitor



Difference between Semaphore and Monitor

Basis for Comparison	Semaphore	Monitor
Basic	Semaphores is an integer variable S.	Monitor is an abstract data type.
Action	The value of Semaphore S indicates the number of shared resources availabe in the system	The Monitor type contains shared variables and the set of procedures that operate on the shared variable.
Access	When any process access the shared resources it perform wait() operation on S and when it releases the shared resources it performs signal() operation on S.	When any process wants to access the shared variables in the monitor, it needs to access it through the procedures.
Condition variable	Semaphore does not have condition variables.	Monitor has condition variables.

➤ Condition Variables

- But Monitors are not powerful enough to model some synchronization schemes.
- So the solution is provided by Condition Variables.

Declaration:

`condition x, y;`

- Two operations on a condition variable:
 - `x.wait ()` – a process that invokes the operation is suspended until `x.signal ()`
 - `x.signal ()` – resumes one of processes (if any) that invoked `x.wait ()`
 - If no `x.wait ()` on the variable, then it has no effect on the variable

Inter process Communication(IPC)

- IPC is the mechanism provided by the operating system that allows processes to communicate with each other.
- This communication involve a process letting another process know that some event occurred or the transferring of data from one process to another.



- Processes within a system may be independent or cooperating.
- Cooperating process can affect or be affected by other processes, including sharing data.
- Reasons for cooperating processes:
 - Information sharing
 - Modularity
 - Computation speedup
 - Convenience
- Cooperating processes need inter process communication(IPC)

- There are several reasons for providing an environment that allows process cooperation:
- **Information sharing.** Since several users may be interested in the same piece of information (for instance, a shared file), we must provide an environment to allow concurrent access to such information.
- **Computation speedup.** If we want a particular task to run faster, we must break it into subtasks, each of which will be executing in parallel with the others. Notice that such a speedup can be achieved only if the computer has multiple processing cores.
- **Modularity.** We may want to construct the system in a modular fashion, dividing the system functions into separate processes or threads
- **Convenience.** Even an individual user may work on many tasks at the same time. For instance, a user may be editing, listening to music, and compiling in parallel.

- Cooperating processes require an **inter process communication (IPC) mechanism** that will allow them to exchange data and information. There are two fundamental models of inter process communication: **shared memory and message passing**.
- In the shared-memory model, a region of memory that is shared by cooperating processes is established. Processes can then exchange information by reading and writing data to the shared region.
- In the message-passing model, communication takes place by means of messages exchanged between the cooperating processes.

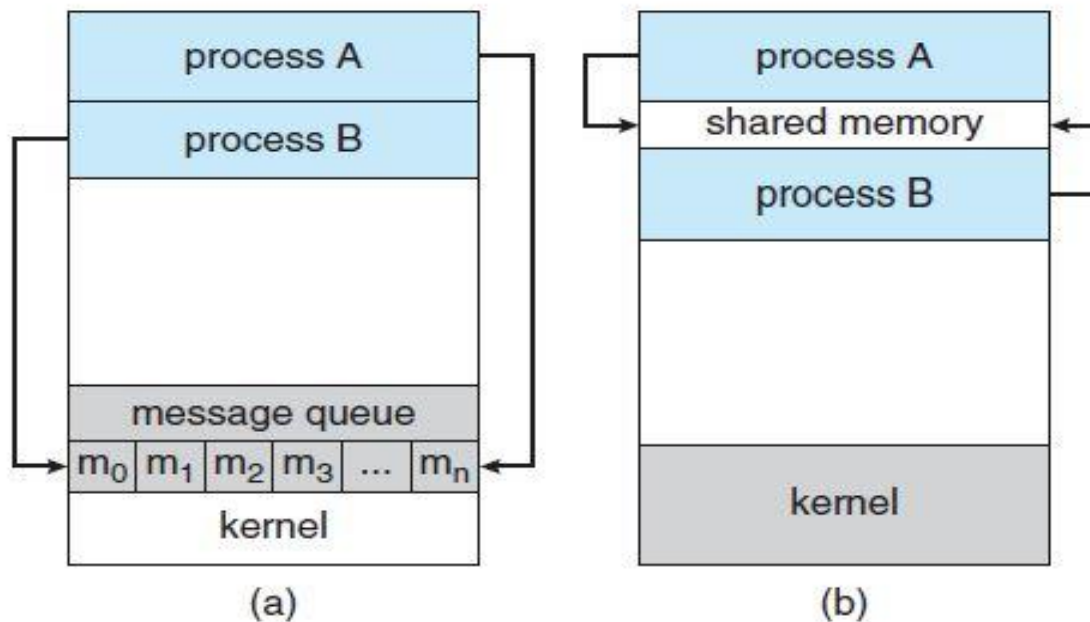


Figure 3.12 Communications models. (a) Message passing. (b) Shared memory.

Shared-Memory Systems

- Inter process communication using shared memory requires communicating processes to establish a region of shared memory.
- Typically, a shared-memory region resides in the address space of the process creating the shared-memory segment.
- Other processes that wish to communicate using this shared-memory segment must attach it to their address space.
- normally, the operating system tries to prevent one process from accessing another process's memory.
- Shared memory requires that two or more processes agree to remove this restriction

Shared-Memory Systems

- let's consider the producer–consumer problem, which is a common paradigm for cooperating processes.
- **A producer process produces information that is consumed by a consumer process.**
- For example, a compiler may produce assembly code that is consumed by an assembler. The assembler, in turn, may produce object modules that are consumed by the loader
- For example, a web server produces (that is, provides) HTML files and images, which are consumed (that is, read) by the client web browser requesting the resource.
- One solution to the producer–consumer problem uses shared memory
- To allow producer and consumer processes to run concurrently, we must have available a buffer of items that can be filled by the producer and emptied by the consumer.
- This buffer will reside in a region of memory that is shared by the producer and consumer processes
- A producer can produce one item while the consumer is consuming another item. The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced

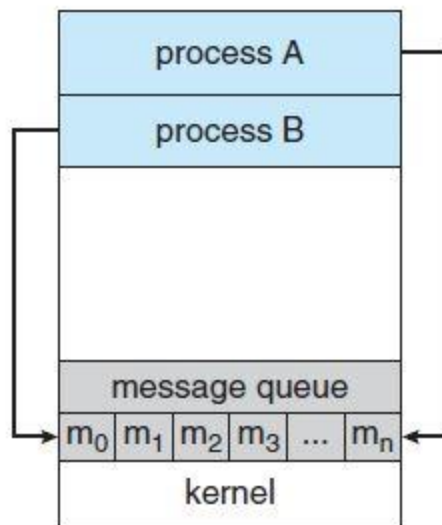
Types of Buffers

- Two types of buffers can be used. The **Unbounded buffer places no practical** limit on the size of the buffer. The consumer may have to wait for new items, but the producer can always produce new items.
- The **bounded buffer assumes** a fixed buffer size. In this case, the consumer must wait if the buffer is empty, and the producer must wait if the buffer is full.

Message-Passing Systems

- Message passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space.
- It is particularly useful in a distributed environment, where the communicating processes may reside on different computers connected by a network.
- A message-passing facility provides at least two operations:
 - `send(message)` `receive(message)`
- Messages sent by a process can be either fixed or variable in size.
- If only **fixed-sized messages** can be sent, the system-level implementation is straightforward. however, makes the task of programming more difficult.
- **variable-sized messages** require a more complex system-level implementation, but the programming task becomes simpler

- If processes *P* and *Q* want to communicate, they must send messages to and receive messages from each other.
 - **communication link must exist between** them. This link can be implemented in a variety of ways
 - Here are several methods for logically implementing a link and the send()/receive() operations:
 - Direct or indirect communication
 - Synchronous or asynchronous communication
 - Automatic or explicit buffering
- There are several issues related with features like:
- Naming
 - Synchronization
 - buffering



(a)

Naming

- Processes that want to communicate must have a way to refer to each other. They can use either direct or indirect communication.
- Under **direct communication**, each process that wants to communicate must explicitly name the recipient or sender of the communication. In this scheme, the `send()` and `receive()` primitives are defined as:
 - `send(P, message)`—Send a message to process P.
 - `receive(Q, message)`—Receive a message from process Q.

Naming

- A communication link in this scheme has the following properties:
 - A link is established automatically between every pair of processes that want to communicate. The processes need to know only each other's identity to communicate.
 - A link is associated with exactly two processes.
 - Between each pair of processes, there exists exactly one link.
- This scheme exhibits ***symmetry in addressing; that is, both the sender*** process and the receiver process must name the other to communicate. A variant of this scheme employs ***asymmetry in addressing. Here, only the sender*** names the recipient; the recipient is not required to name the sender. In this scheme, the send() and receive() primitives are defined as follows:
 - send(P, message)—Send a message to process P.
 - receive(id, message)—Receive a message from any process. The variable id is set to the name of the process with which communication has taken place.

- With ***indirect communication***, the messages are sent to and received from mailboxes, or ports.
- A mail box can be viewed abstractly as an object into which messages can be placed by processes and from which messages can be removed.
- Each mailbox has a unique identification
- two processes can communicate only if they have a shared mailbox. The send() and receive() primitives are defined as follows:
 - send(A, message)—Send a message to mailbox A.
 - receive(A, message)—Receive a message from mailbox A.

- In this scheme, a communication link has the following properties:
 - A link is established between a pair of processes only if both members of the pair have a shared mailbox.
 - A link may be associated with more than two processes.
 - Between each pair of communicating processes, a number of different links may exist, with each link corresponding to one mailbox.

- Now suppose that processes *P1*, *P2*, and *P3* all share mailbox *A*. Process *P1* sends a message to *A*, while both *P2* and *P3* execute a *receive()* from *A*. Which process will receive the message sent by *P1*? The answer depends on which of the following methods we choose:
 - Allow a link to be associated with two processes at most.
 - Allow at most one process at a time to execute a *receive()* operation.
 - Allow the system to select arbitrarily which process will receive the message (that is, either *P2* or *P3*, but not both, will receive the message). The system may define an algorithm for selecting which process will receive the message (for example, **round robin, where processes take turns receiving** messages). The system may identify the receiver to the sender.

Synchronization

- Communication between processes takes place through calls to `send()` and `receive()` primitives. There are different design options for implementing each primitive. Message passing may be either **blocking or non blocking** — also known as **synchronous and asynchronous**.
 - **Blocking send.** The sending process is blocked until the message is received by the receiving process or by the mailbox.
 - **Non blocking send.** The sending process sends the message and resumes operation.
 - **Blocking receive.** The receiver blocks until a message is available.
 - **Non blocking receive:** The receiver retrieves either a valid message or a null.

Buffering

- Whether communication is direct or indirect, messages exchanged by communicating processes reside in a temporary queue.
- Basically, such queues can be implemented in three ways:
 1. **Zero capacity.** The queue has a maximum length of zero; thus, the link cannot have any messages waiting in it. In this case, the sender must block until the recipient receives the message.
 2. **Bounded capacity.** The queue has finite length n ; *thus, at most n messages* can reside in it. If the queue is not full when a new message is sent, the message is placed in the queue (either the message is copied or a pointer to the message is kept), and the sender can continue execution without waiting. The link's capacity is finite, however. If the link is full, the sender must block until space is available in the queue.
 3. **Unbounded capacity.** The queue's length is potentially infinite; thus, any number of messages can wait in it. The sender never blocks.

Pipes

- A pipe is a data channel that is unidirectional. Two pipes can be used to create a two way data channel between two processes.

Message passing:

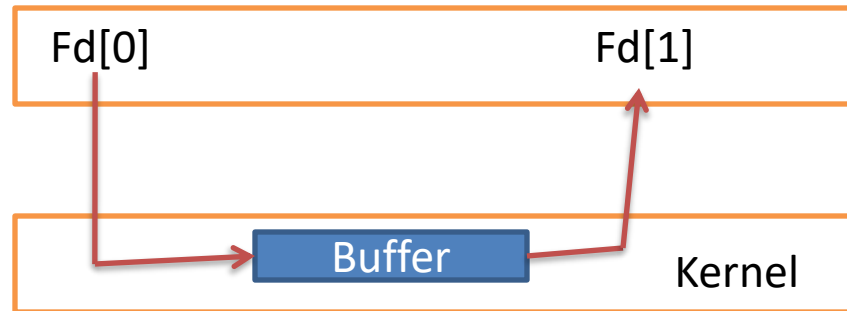
- It is a mechanism for a process to communicate and synchronize.
- Using message passing, the process communicates with each other without resorting to shared variables.



Pipe is accessed through `int fd[]` file descriptors:

- `Fd[0]` is read end
- `Fd[1]` is write end

- Pipes can be accessed using read() and write() system calls



Pipe Creation:

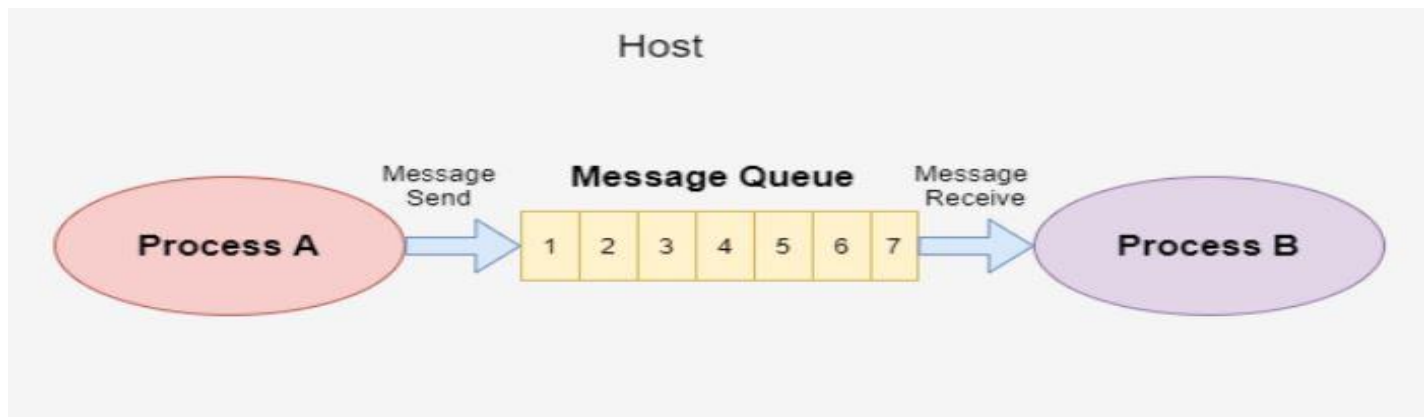
- `[int pipe (int fds[2])]`
- `Fd[0] & Fd[1]`
- `Read     write`
- Return '0' success
- '-1' Failure
- It is FIFO
- At a time in pipe we can write 512 bytes of data.
- At a time in pipe we can read 1 byte of data.

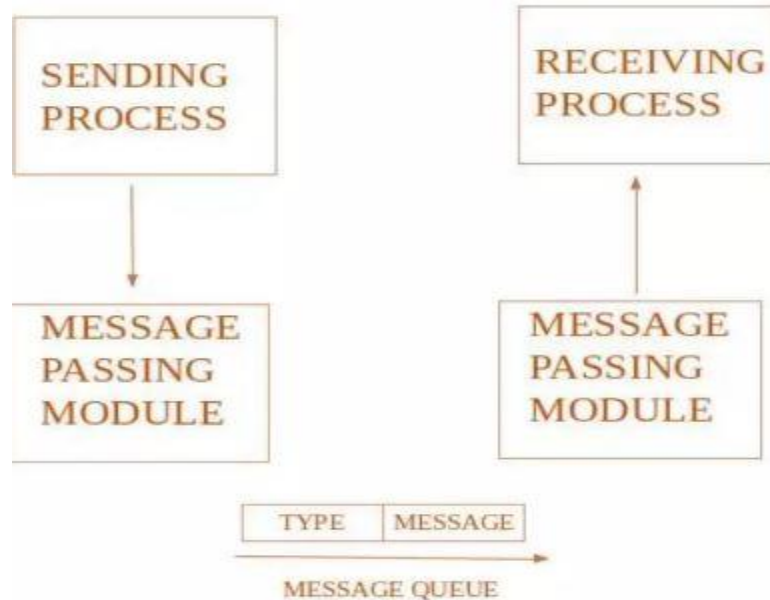
Message queues

- A message queue is a linked list of messages stored within the kernel.
- Message queues are one of the inter-process communication mechanism which allows processes to exchange data in the form of messages between two processes.
- It allows processes to communicate asynchronously by sending messages to each other where the messages are stored in a queue, waiting to be processed, and are deleted after being processed.

IPC using message queue involves the following steps:

- Defining a Message Structure
- Creating the Message Queue
- Sending the Message to the Process A through the Message Queue
- Receiving the Message from the Process A through the Message Queue





System calls used for message queues:

- **ftok()**: is use to generate a unique key.
- **msgget()**: either returns the message queue identifier for a newly created message queue or returns the identifiers for a queue which exists with the same key value.
- **msgsnd()**: Data is placed on to a message queue by calling msgsnd().
- **msgrcv()**: messages are retrieved from a queue.
- **msgctl()**: It performs various operations on a queue. Generally it is use to destroy message queue.

FIFO

- FIFOs(first in first out) are a variation on a pipe that creates a more flexible communication structure.
- FIFOs work by attaching a filename to the pipe. For this reason, FIFOs are also called named pipes.
- Named pipes can be used to provide communication between processes on the same computer or between processes on different computers across a network.
- FIFOs are created by one process that calls mkfifo().

